

Projekt-Spezifikation: School Gamification Platform ("EduRank")

1. Projektübersicht

Entwicklung einer Web-Applikation, die Schulaufgaben durch Gamification-Elemente anreichert. Der Kernprozess besteht darin, dass Nutzer analoge oder digitale Aufgaben hochladen, diese von einer KI in ein interaktives Format umgewandelt werden und das Lösen dieser Aufgaben spielerische Belohnungen (XP, Ranking, Level) einbringt.

****Ziel:**** Hausaufgaben und Lernen durch RPG- und Wettbewerbs-Mechaniken attraktiv zu machen.

2. Technischer Stack (Vorschlag für den Agenten)

- * ****Backend:**** Laravel (PHP) - robustes User-Management, Eloquent ORM für komplexe Relationen (Ranks, History).
- * ****Frontend:**** Vue.js oder React (SPA) oder Laravel Livewire - für reaktive UI (wichtig für Timer und Realtime-PvP).
- * ****KI-Integration:**** OpenAI API (GPT-4o) oder Anthropic API (Claude 3.5 Sonnet) mit Vision-Support.
- * ****Datenbank:**** MySQL/PostgreSQL.
- * ****Realtime:**** Pusher oder Laravel Reverb (für den Ranked PvP Modus).

3. Core Feature: Der "Task-Engine" Workflow

A. Upload & Parsing (Der "Dungeon Generator")

1. ****Input:**** User lädt Bild (Foto vom Arbeitsblatt) oder Text einer Schulaufgabe hoch.
2. ****Processing (KI-Worker):****
 - * OCR (Texterkennung).
 - * ****Semantische Analyse:**** Die KI muss folgende Metadaten extrahieren:
 - * ***Fach:*** (Mathe, Bio, Geschichte...).
 - * ***Thema:*** (z.B. Integralrechnung, Französische Revolution).
 - * ***Schwierigkeitsgrad:*** (1-10, geschätzt durch KI).
 - * ***Aufgabentyp:*** (Multiple Choice, Freitext, Rechenweg, Lückentext).
 - * ****Lösungs-Generierung:**** Die KI generiert im Hintergrund die ***Musterlösung*** und speichert diese verschlüsselt in der DB (für die spätere Validierung).
3. ****Output:**** Die Aufgabe wird dem User in einem sauberen UI ("Game Card") präsentiert.

B. Solving Interface (Die "Arena")

- * ****Darstellung:**** Markdown/LaTeX Support für Formeln.
- * ****Eingabefeld:**** Code-Editor-ähnliches Feld für Text oder Formeln.
- * ****Validierung:**** User sendet Lösung ab -> KI (oder Regex bei simplen Antworten) vergleicht User-Input mit Musterlösung.
 - * ***Match:*** Punktevergabe.
 - * ***Fail:*** Feedback-Loop (Tipp geben gegen Währung, s.u.).

4. Gamification-Systeme & Mechaniken

A. Progression & RPG-Elemente

- * ****Global Level & XP:**** Klassische XP-Kurve. Je schwieriger die Aufgabe (KI-Rating), desto mehr XP.
- * ****Fach-Spezifische Skill-Trees:****
 - * Statt nur einem Level gibt es "Mastery" pro Fach (z.B. "Level 12 Pyromancer" für Chemie, "Level 5 Scribe" für Deutsch).
 - * Visueller Skill-Tree, der sich füllt, je mehr Aufgaben zu einem Topic gelöst werden.
- * ****Streaks:**** Multiplikator für tägliches Einloggen/Lösen (z.B. "Scholar's Momentum").

B. Währung & Economy

- * ****"Brain Coins" (Währung):**** Verdient durch Lösen von Aufgaben.
- * ****Shop:****
 - * ***Kosmetisch:*** Profilrahmen, Avatar-Skins, Titel (z.B. "Mathe-Magier").
 - * ***Funktional:*** "Hint Potions" (KI gibt einen Tipp), "Time Freeze" (im Zeit-Modus 10s Pause).

C. Leaderboards (Ranglisten)

- * ****Global:**** All-Time XP.
- * ****Weekly League:**** Ähnlich wie Duolingo. Auf- und Abstieg in Ligen (Bronze, Silber, Gold, Diamant, Challenger).
- * ****Topic Legends:**** "Bester Spieler in 'Stochastik' diese Woche".

5. Spiel-Modi

Modus 1: "Adventure Mode" (Solo, Standard)

- * Der User löst seine eigenen hochgeladenen Aufgaben.
- * Kein Zeitdruck (außer optional).
- * Fokus: Lernen und Verstehen.

Modus 2: "Time Attack" (Solo, Arcade)

- * User wählt ein Fach.
- * KI generiert 5 Kurze Fragen (aus einem Pool oder basierend auf früheren Uploads).
- * Ziel: So schnell wie möglich lösen.
- * Highscore-basiert.

Modus 3: "Ranked Duel" (PvP, Synchron)

- * ****Matchmaking:**** Spieler A (Level 10) sucht Gegner. Spieler B (Level 11) wird gefunden.
- * ****Szenario:**** Das System generiert für beide **dieselbe** KI-Aufgabe (z.B. "Löse diese quadratische Gleichung").
- * ****Win-Condition:**** Wer zuerst die korrekte Lösung eingibt, gewinnt.
- * ****Elo-System:**** Gewinner kriegt Ranked Points (RP), Verlierer verliert RP.

Modus 4: "Boss Raid" (Koop oder Solo, Highlight)

- * Eine sehr komplexe Aufgabe (z.B. eine ganze Klausur-Aufgabe) wird als "Boss" dargestellt.
- * Der Boss hat "HP" (Health Points). Jeder Teilschritt der Aufgabe, der richtig gelöst wird, zieht dem Boss HP ab.
- * Macht man Fehler, nimmt der Spieler Schaden (Zeitstrafe oder Punkteabzug).

6. Datenbank-Modell (Entwurf für Agenten)

- * `users`: id, name, email, level, xp, elo_rating, coins
- * `subjects`: id, name (Math, Bio, etc.)
- * `tasks`: id, user_id, raw_content, formatted_content, solution_hash, difficulty, type, status (pending, solved)
- * `user\_subject\_mastery`: user_id, subject_id, level, xp (für Fach-Levels)
- * `matches`: id, player_1, player_2, winner_id, task_data, status (für Ranked)

7. UI/UX Design-Richtlinien

- * **Theme:** "Dark Academia" meets "Cyberpunk" oder "Clean Modern" (konfigurierbar).
- * **Feedback:** Jede Aktion muss visuelles Feedback geben (Partikeleffekte bei Level Up, Sound bei richtiger Antwort).
- * **Mobile First:** Das UI muss auf dem Handy funktionieren (für schnelles Lernen im Bus).

8. Nächste Schritte (Agenten-Instruktionen)

1. Setze ein neues Laravel Projekt auf.
2. Erstelle die Migrationen für User und Tasks.
3. Implementiere den "KI-Service", der einen Mockup-Text an die OpenAI API sendet und JSON zurückbekommt.
4. Erstelle das Frontend-Formular für den Upload.